

计算机科学与技术学院神经网络与深度学习课程实验报告

实验题目：KNN 实现鸢尾花分类实验		学号：202300130183
日期：2025/9/18	班级：2023 级智能班	姓名：宋浩宇
Email: zhitian0420@gmail		
实验目的： 1.理解 KNN（K-Nearest Neighbors）算法的基本原理和工作机制。 2.掌握使用 KNN 算法对鸢尾花数据集进行分类的方法。 3.学会使用准确率等指标评估分类模型的性能。 4.掌握数据集划分方法及 k 折交叉验证技术。 5.理解验证集在模型调参中的作用，掌握通过交叉验证选择最优 K 值的方法。		
实验软件和硬件环境： 软件环境： 主系统：Windows 11 家庭中文版 23H2 ; ArchLinux 编辑器：Visual Studio Code ; PyCharm 2025.1.2 Python 解释器：Python 3.9 Pytorch 版本：2.8.0 CUDA 版本：12.8.97 硬件环境： CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz 内存：32.0 GB (31.6 GB 可用) 磁盘驱动器：NVMe WD_BLACKSN850X2000GB 显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU CUDA 核心数：7424		
实验原理和方法： 实验原理为 KNN 算法，使用的计算最优 K 值的方法为交叉验证法。		
实验步骤：（不要求罗列完整源代码） 首先导入并划分数据集： <pre># 下载/加载数据集 iris = load_iris() # 划分训练集和测试集 x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2, random_state=11)</pre> 然后实现 KNN 算法： <pre># 实现 KNN 模型 class KNN(object): def __init__(self, n_neighbors=1, dist_func=l1_distance): self.n_neighbors = n_neighbors</pre>		

```

        self.dist_func = dist_func
        self.x = None
        self.y = None
    def fit(self, x_train, y_train):
        self.x = x_train
        self.y = y_train
        pass
    def predict_one(self, x_test):
        if self.x is None or self.y is None:
            raise Exception("请先调用 fit 方法")
        distances = []
        for i in self.x:
            distances.append(self.dist_func(i,x_test))
        sorted_index = np.argsort(distances)
        # print(distances[sorted_index[:self.n_neighbors]])
        result = {}
        for i in range(self.n_neighbors):
            if self.y[sorted_index[i]] not in result:
                result[self.y[sorted_index[i]]] = 1
            else:
                result[self.y[sorted_index[i]]] += 1
        # print(np.array(distances)[sorted_index])
        # print(result)
        return max(result, key=result.get)
    def predict(self, x_test):
        y_pred = []
        for i in x_test:
            y_pred.append(self.predict_one(i))
        return y_pred

```

然后通过交叉验证寻找最优 K 值:

```

kf = sklearn.model_selection.KFold(n_splits=5, shuffle=True,
random_state=35)
k_range = range(1, 11)
cv_scores = []
for k in k_range:
    KNN_model = KNN(n_neighbors=k)
    print("K:", k)
    fold_scores = []
    for train_index, test_index in kf.split(x_train):
        x_train_fold, x_test_fold = x_train[train_index],
x_train[test_index]
        y_train_fold, y_test_fold = y_train[train_index],
y_train[test_index]
        print("训练集大小:", len(x_train_fold), "测试集大小:",

```

```
len(x_test_fold))
    print("训练集标签:", y_train_fold)
    print("测试集标签:", y_test_fold)
    KNN_model.fit(x_train_fold, y_train_fold)
    y_pred = KNN_model.predict(x_test_fold)
    print("预测结果:", y_pred)
    fold_scores.append(accuracy_score(y_test_fold, y_pred))
print(fold_scores)
cv_scores.append(np.mean(fold_scores))
print(cv_scores)
best_k = np.argmax(cv_scores) + 1
print("最佳 k 值:", best_k)
```

然后我们就可以获取最优 K 值下的模型并进一步完成测试了

```
best_model = KNN(n_neighbors=best_k)
best_model.fit(x_train, y_train)
y_pred = best_model.predict(x_test)
print("测试集准确率:", accuracy_score(y_test, y_pred))
```

最终结果是:

最佳 k 值: 3

测试集准确率: 0.9333333333333333

结论分析与体会:

从结果上来看, 对于 iris 这个数据集, 使用 KNN 方法进行预测, 最优的 K 值就是 3, 我重复跑了很多次, 都是这个结果, 这个结果的可信度很高。从预测准确率上来看, KNN 能够较好的解决 iris 数据集上的预测这个问题, 且成功率较高, 另外也再次表明我们通过交叉验证获得的最优 K 值确实能得到较好的模型性能。

就实验过程中遇到和出现的问题, 你是如何解决和处理的, 自拟 1—3 道问答题:

问题 1: 没有使用过 sklearn 这个库, 对于一些函数的使用不是很清楚。

解答 1: 通过查看源代码中的注释和官方 api 文档确认使用方法。